# ANEXO FAKETIME

📙Descargar PDF


- ANEXO FAKETIME
  - INTRODUCCIÓN
  - INSTALACIÓN
  - FUNCIONAMIENTO GENERAL
  - OPCIONES DE LÍNEA DE COMANDOS
  - PRIMEROS EJEMPLOS
  - FORMATOS DE TIMESTAMP
    - FORMATO SIMPLE DEL WRAPPER
    - FORMATO AVANZADO CON `-f`
      - FECHA ABSOLUTA FIJA
      - DESPLAZAMIENTO RELATIVO
      - FORMATO START-AT
      - FRACCIONES
      - MODIFICADOR `x` PARA ACELERAR O FRENAR EL RELOJ
      - MODIFICADOR `i` PARA INCREMENTOS POR LLAMADA
  - VARIABLES DE ENTORNO PRINCIPALES
    - VARIABLES BÁSICAS DE CONFIGURACIÓN
    - VARIABLES DE COMPATIBILIDAD Y ESTABILIDAD
    - VARIABLES PARA SEGUIR UN FICHERO O UNA REFERENCIA EXTERNA
    - VARIABLES PARA ACTIVACIÓN LIMITADA EN EL TIEMPO O POR NÚMERO DE LLAMADAS
    - VARIABLES PARA FILTRAR PROCESOS
    - VARIABLES PARA EJECUTAR PROCESOS EXTERNOS
    - VARIABLES PARA GUARDAR, CARGAR O COMPARTIR ESTADOS TEMPORALES
  - EJEMPLOS PRÁCTICOS
  - LIMITACIONES Y ADVERTENCIAS
  - CRITERIO DE USO
  - REFERENCIAS

# INTRODUCCIÓN

`faketime` es la utilidad de línea de comandos incluida en el proyecto **libfaketime**. Su finalidad es ejecutar un programa haciendo que ese programa "vea" una fecha y una hora distintas de las reales, sin modificar el reloj global del sistema.

Según la documentación oficial, la herramienta actúa como un *wrapper* de `libfaketime`: prepara las variables necesarias para que el enlazador cargue una biblioteca mediante `LD_PRELOAD`, y esa biblioteca intercepta llamadas relacionadas con el tiempo y con marcas temporales del sistema de archivos, como `time()` o `fstat()`.

Por tanto, al usar `faketime` no se cambia la hora del sistema, sino únicamente la percepción temporal del proceso lanzado y, en muchos casos, la de sus procesos hijo.

# INSTALACIÓN

En sistemas Debian y derivados, la utilidad suele instalarse mediante el paquete `faketime`.

**Instalación básica**

El siguiente ejemplo instala la utilidad y deja disponible tanto el comando `faketime` como la biblioteca `libfaketime`.

```
apt update
apt install faketime
```

# FUNCIONAMIENTO GENERAL

La sintaxis general descrita en la página de manual es la siguiente:

```
faketime [options] timestamp program [arguments...]
```

Los tres elementos fundamentales son estos:

- Un conjunto opcional de **opciones** del envoltorio `faketime`.
- Un **timestamp** que indica qué tiempo debe simularse.
- El **programa** que se ejecutará con esa percepción temporal alterada.

La documentación oficial distingue dos formas principales de indicar el tiempo falso:

- El **formato simple**, que usa por defecto el *wrapper* y delega el análisis de la fecha en una utilidad compatible con GNU `date`.
- El **formato avanzado**, activado con `-f`, que pasa la especificación directamente a `libfaketime` y permite usar desplazamientos relativos, fechas de arranque, aceleración del reloj y otras funciones avanzadas.

# OPCIONES DE LÍNEA DE COMANDOS

Estas son las opciones documentadas oficialmente para el comando `faketime`:

| Opción | Función | Observaciones |
|---|---|---|
| `--help` | Muestra la ayuda y termina. | No ejecuta ningún programa. |
| `--version` | Muestra la versión y termina. | Útil para comprobar la versión instalada. |
| `-m` | Usa la variante multihilo de `libfaketime`. | Recomendable cuando el programa objetivo usa varios hilos. |
| `-p PID` | Hace creer al programa que su PID es el indicado. | Solo está disponible si se compiló con `FAKE_PID`. |
| `-f` | Activa el formato avanzado de timestamp. | Es la opción clave para usar desplazamientos relativos, `@`, `x` o `i`. |
| `--exclude-monotonic` | Evita falsear llamadas a `clock_gettime()` con `CLOCK_MONOTONIC`. | Puede ser necesario para evitar bloqueos o comportamientos anómalos. |
| `--date-prog PATH` | Indica qué utilidad compatible con GNU `date` debe usar el *wrapper* para interpretar fechas en formato simple. | Solo afecta al formato simple del *wrapper*. |

# PRIMEROS EJEMPLOS

### Fecha legible en lenguaje natural

El siguiente ejemplo usa el formato simple del *wrapper* para ejecutar `date` como si fuera el viernes anterior a las 17:00.

```
faketime 'last Friday 5 pm' /bin/date
```

### Fecha absoluta simple

El siguiente ejemplo ejecuta `date` con una fecha y hora concretas interpretadas por el *wrapper*.

```
faketime '2018-12-24 08:15:42' /bin/date
```

### Desplazamiento relativo avanzado

El siguiente ejemplo adelanta dos horas la percepción temporal del programa usando el formato avanzado de `libfaketime`.

```
faketime -f '+2h' /bin/date
```

# FORMATOS DE TIMESTAMP

## FORMATO SIMPLE DEL WRAPPER

Cuando no se usa `-f`, `faketime` interpreta la fecha mediante una utilidad compatible con GNU `date`. Esto permite escribir expresiones cómodas como `yesterday`, `tomorrow 08:00`, `last monday` o una fecha completa en formato legible.

Es el modo más cómodo para pruebas rápidas, pero no expone todas las capacidades de `libfaketime`.

## FORMATO AVANZADO CON `-f`

La documentación oficial recomienda conocer este formato porque es el que da acceso a las funciones realmente potentes de la herramienta.

### FECHA ABSOLUTA FIJA

El formato exacto es:

```
YYYY-MM-DD hh:mm:ss
```

Esta modalidad **congela** el reloj en el instante indicado. Si el programa consulta la hora dos veces con cinco minutos de diferencia real, seguirá viendo la misma hora falsa.

La documentación insiste en que esta opción puede romper aplicaciones que esperan que el tiempo avance, por lo que debe usarse con cautela.

**Reloj congelado en una fecha fija**

El siguiente ejemplo hace que `date` vea siempre la misma fecha absoluta.

```
faketime -f '2026-02-16 10:00:00' /bin/date
```

### DESPLAZAMIENTO RELATIVO

El formato general es:

```
[+/-]cantidad[m/h/d/y]
```

Si no se indica sufijo, la cantidad se interpreta en segundos. Los sufijos documentados son:

- `m` para minutos.

- `h` para horas.
- `d` para días.
- `y` para años de 365 días.

Este es el modo más habitual, porque desplaza la hora respecto a la hora real y deja que siga avanzando normalmente.

**Retroceder diez minutos**

El siguiente ejemplo ejecuta un comando como si estuviera diez minutos en el pasado.

```
faketime -f '-10m' /bin/date
```

**Adelantar catorce días**

El siguiente ejemplo ejecuta un comando como si estuviera dos semanas en el futuro.

```
faketime -f '+14d' /bin/date
```

## FORMATO START-AT

El formato es:

```
@YYYY-MM-DD hh:mm:ss
```

En este caso el reloj empieza en la fecha indicada, pero sigue corriendo. Es decir, sirve para arrancar un proceso en un instante concreto sin congelar el tiempo.

**Arrancar en una fecha concreta sin congelar el reloj**

El siguiente ejemplo inicia una shell en una fecha concreta y deja que el tiempo avance a partir de ella.

```
faketime -f '@2026-02-16 10:00:00' /bin/bash
```

## FRACCIONES

La documentación oficial indica que pueden utilizarse fracciones en desplazamientos y modificadores. El separador decimal depende de la configuración regional, por lo que puede ser necesario usar coma o punto.

**Desplazamiento fraccional**

El siguiente ejemplo adelanta la hora simulada una hora y media.

```
faketime -f '+1,5h' /bin/date
```

### MODIFICADOR `x` PARA ACELERAR O FRENAR EL RELOJ

El modificador `x` cambia la velocidad a la que el programa percibe el paso del tiempo. Debe ir acompañado de un desplazamiento previo, aunque este sea `+0`.

**Reloj diez veces más rápido**

El siguiente ejemplo hace que una shell crea que el tiempo avanza diez veces más deprisa que en el sistema real.

```
faketime -f '+0 x10' /bin/bash -c 'while true; do echo $SECONDS; sleep 1; done'
```

**Reloj más lento**

El siguiente ejemplo hace que el programa perciba el tiempo al 20 por ciento de su velocidad habitual.

```
faketime -f '-7d x0,2' /bin/bash -c 'while true; do echo $SECONDS; sleep 1; done'
```

### MODIFICADOR `i` PARA INCREMENTOS POR LLAMADA

El modificador `i` hace que el tiempo simulado aumente un intervalo fijo en cada llamada relacionada con el tiempo, con independencia del reloj real. La documentación lo presenta como útil para pruebas deterministas.

**Incremento fijo por llamada**

El siguiente ejemplo avanza dos segundos simulados por cada llamada relevante al tiempo.

```
faketime -f '+0 i2,0' /bin/bash -c 'while true; do date; sleep 1; done'
```

# VARIABLES DE ENTORNO PRINCIPALES

Aunque el comando `faketime` simplifica mucho el uso de la biblioteca, buena parte de la funcionalidad oficial se controla mediante variables de entorno de `libfaketime`.

## VARIABLES BÁSICAS DE CONFIGURACIÓN

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME` | Define directamente el tiempo simulado. | Tiene prioridad sobre los ficheros de configuración. |

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_FMT` | Cambia el formato usado para interpretar `FAKETIME`. | Permite, por ejemplo, usar marcas de tiempo Unix con `%s`. |
| `FAKETIME_TIMESTAMP_FILE` | Lee la especificación temporal desde un fichero. | Útil para cambiar el tiempo simulado mientras el programa sigue ejecutándose. |
| `FAKETIME_UPDATE_TIMESTAMP_FILE` | Actualiza el fichero indicado en `FAKETIME_TIMESTAMP_FILE` cuando se interceptan llamadas de cambio de hora. | Requiere compatibilidad de compilación con interceptación de funciones de ajuste del reloj. |
| `HOME/.faketimerc` | Fichero por usuario con una sola línea equivalente a `FAKETIME`. | Se usa si no se ha definido `FAKETIME_TIMESTAMP_FILE`. |
| `/etc/faketimerc` | Valor por defecto de sistema. | Solo se usa si no hay fichero de usuario ni `FAKETIME_TIMESTAMP_FILE`. |

## VARIABLES DE COMPATIBILIDAD Y ESTABILIDAD

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_DONT_FAKE_MONOTONIC` | Evita falsear el reloj monotónico. | Muy importante en algunos entornos Java/JVM. |
| `NO_FAKE_STAT` | Desactiva el falseado de marcas temporales del sistema de archivos. | Útil si esa interceptación confunde a la aplicación. |
| `FAKETIME_DONT_RESET` | Evita que ciertos procesos hijo reinicien el tiempo de tipo *start-at*. | Resulta relevante en cadenas de procesos. |
| `FAKETIME_CACHE_DURATION` | Ajusta cada cuántos segundos se releen los ficheros de configuración. | El valor por defecto documentado es 10 segundos. |
| `FAKETIME_NO_CACHE` | Desactiva la caché de configuración. | Mejora la reactividad, pero puede penalizar el rendimiento. |
| `FAKETIME_FORCE_MONOTONIC_FIX` | Activa un ajuste para ciertos problemas con `CLOCK_MONOTONIC`. | Debe usarse solo si realmente hace falta. |
| `FAKETIME_DISABLE_SHM` | Desactiva el uso de memoria compartida. | Equivale al modificador `--disable-shm` mencionado en el README de la biblioteca. |

## VARIABLES PARA SEGUIR UN FICHERO O UNA REFERENCIA EXTERNA

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_FOLLOW_FILE` | Usa la marca temporal de un fichero como origen del tiempo simulado. | Se activa definiendo `FAKETIME='%'`. |
| `FAKETIME_FOLLOW_ABSOLUTE` | Hace que el tiempo solo avance cuando avance el timestamp del fichero seguido. | Es un submodo especial de `FAKETIME_FOLLOW_FILE`. |
| `FAKETIME_XRESET` | Suaviza cambios en el modificador `x` durante la ejecución. | La documentación la considera experimental. |

## VARIABLES PARA ACTIVACIÓN LIMITADA EN EL TIEMPO O POR NÚMERO DE LLAMADAS

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_START_AFTER_SECONDS` | Empieza a falsear el tiempo después de cierto número de segundos. | El valor `-1` significa ignorar el límite. |
| `FAKETIME_STOP_AFTER_SECONDS` | Deja de falsear el tiempo tras cierto número de segundos. | Puede combinarse con la variable anterior. |
| `FAKETIME_START_AFTER_NUMCALLS` | Activa el falseado tras cierto número de llamadas relacionadas con el tiempo. | Orientado a pruebas muy específicas. |
| `FAKETIME_STOP_AFTER_NUMCALLS` | Desactiva el falseado tras cierto número de llamadas. | También es un caso avanzado. |

## VARIABLES PARA FILTRAR PROCESOS

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_ONLY_CMDS` | Solo aplica `faketime` a los comandos indicados. | Los nombres se separan por comas. |
| `FAKETIME_SKIP_CMDS` | Excluye del falseado los comandos indicados. | Es mutuamente excluyente con `FAKETIME_ONLY_CMDS`. |

## VARIABLES PARA EJECUTAR PROCESOS EXTERNOS

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_SPAWN_TARGET` | Comando externo que debe lanzarse. | Se ejecuta durante una llamada relacionada con el tiempo del proceso principal. |
| `FAKETIME_SPAWN_SECONDS` | Lanza el comando externo después de cierto número de segundos. | La ejecución real puede retrasarse hasta la siguiente llamada temporal. |

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_SPAWN_NUMCALLS` | Lanza el comando tras cierto número de llamadas temporales. | Alternativa basada en llamadas en vez de tiempo real. |

## VARIABLES PARA GUARDAR, CARGAR O COMPARTIR ESTADOS TEMPORALES

| Variable | Función | Observaciones |
|---|---|---|
| `FAKETIME_SAVE_FILE` | Guarda timestamps falseados en un fichero. | Requiere usar el *wrapper* `faketime`. |
| `FAKETIME_LOAD_FILE` | Reproduce timestamps desde un fichero guardado. | Tras agotar el fichero vuelve a la regla definida en `FAKETIME`. |
| `FAKERANDOM_SEED` | Sustituye ciertos números aleatorios por secuencias deterministas. | Requiere compilación con soporte `FAKE_RANDOM` y no debe usarse en producción. |

# EJEMPLOS PRÁCTICOS

### Probar rápidamente una fecha legible

El siguiente ejemplo resulta útil para comprobar cómo respondería un programa en un momento fácilmente expresable en lenguaje natural.

```
faketime 'tomorrow 08:00' /bin/date
```

### Probar un certificado o una caducidad futura

El siguiente ejemplo adelanta un año la fecha visible para una herramienta de prueba.

```
faketime -f '+1y' /usr/bin/openssl x509 -in cert.pem -noout -dates
```

### Simular una hora concreta para un servicio en primer plano

El siguiente ejemplo ejecuta `sshd` como si fuera una hora determinada, algo útil cuando se quieren probar restricciones horarias configuradas con PAM.

```
faketime -m '2026-02-16 10:00' /usr/sbin/sshd -D -e
```

### Evitar problemas con aplicaciones Java

El siguiente ejemplo muestra el patrón recomendado por la documentación oficial cuando el programa objetivo usa JVM.

```
FAKETIME_DONT_FAKE_MONOTONIC=1 faketime -f '-15d' java -version
```

### Cambiar la especificación temporal desde un fichero

El siguiente ejemplo usa un fichero de configuración que puede modificarse mientras el programa sigue ejecutándose.

```
echo '+0 x1' > /tmp/my-faketime.rc
LD_PRELOAD=libfaketime.so.1 FAKETIME_TIMESTAMP_FILE=/tmp/my-faketime.rc FAKETIME_NO_CACHE=1 ./mi-programa
```

### Seguir la marca temporal de un fichero

El siguiente ejemplo usa la fecha de modificación de un archivo como referencia para el tiempo simulado.

```
touch -t 2602161000.00 /tmp/reloj-demo
LD_PRELOAD=libfaketime.so.1 FAKETIME='%' FAKETIME_FOLLOW_FILE=/tmp/reloj-demo FAKETIME_DONT_RESET=1 /bin/bash
```

# LIMITACIONES Y ADVERTENCIAS

La documentación oficial señala varias limitaciones que conviene conocer antes de usar la herramienta en pruebas reales:

- No funciona con binarios **estáticos** ni con binarios **setuid** debido a las limitaciones de `LD_PRELOAD`.
- Algunos programas cargan bibliotecas del sistema dinámicamente en tiempo de ejecución y pueden esquivar `libfaketime`.
- Algunas aplicaciones Java o basadas en JVM pueden bloquearse si no se excluye el reloj monotónico.
- El falseado de tiempo en procesos hijo suele funcionar, pero las aceleraciones y desaceleraciones con `x` pueden reinicializarse en cada proceso hijo.
- El uso de fechas absolutas congeladas puede romper programas que esperan que el tiempo avance.
- El uso de un reloj simulado a nivel global mediante `/etc/ld.so.preload` es posible, pero la propia documentación advierte de efectos secundarios importantes.

# CRITERIO DE USO

Como criterio general, la documentación de `libfaketime` recomienda preferir **desplazamientos relativos** o **fechas de inicio con** `@` frente a fechas absolutas congeladas. Para pruebas rápidas, el *wrapper* `faketime` suele ser suficiente. Para escenarios complejos, como cambios dinámicos del tiempo, seguimiento de ficheros, límites por llamadas o integración entre varios procesos, conviene trabajar directamente con las variables de entorno de `libfaketime`.

# REFERENCIAS

- Página de manual de `faketime`: https://manpages.debian.org/bookworm/faketime/faketime.1.en.html
- README oficial de `libfaketime`: https://github.com/wolfcw/libfaketime